

CAP 4. CAMADA DE APLICAÇÃO

AULA 7: HTTP – PEDIDOS

INE5422 REDES DE COMPUTADORES II

PROF. ROBERTO WILLRICH (INE/UFSC)

ROBERTO.WILLRICH@UFSC.BR

[HTTPS://MOODLE.UFSC.BR](https://moodle.ufsc.br)

Web: Jargões

Quase todas as páginas WWW consistem de:

- página base HTML, e
- vários objetos referenciados.
- endereçados por uma URL

URL tem duas partes: nome de hospedeiro, e nome de caminho:

`www.someschool.edu/someDept/pic.gif`

Nome do hospedeiro

Nome do caminho

Agente usuário para WWW é o browser:

- Chrome
- Firefox
- Edge

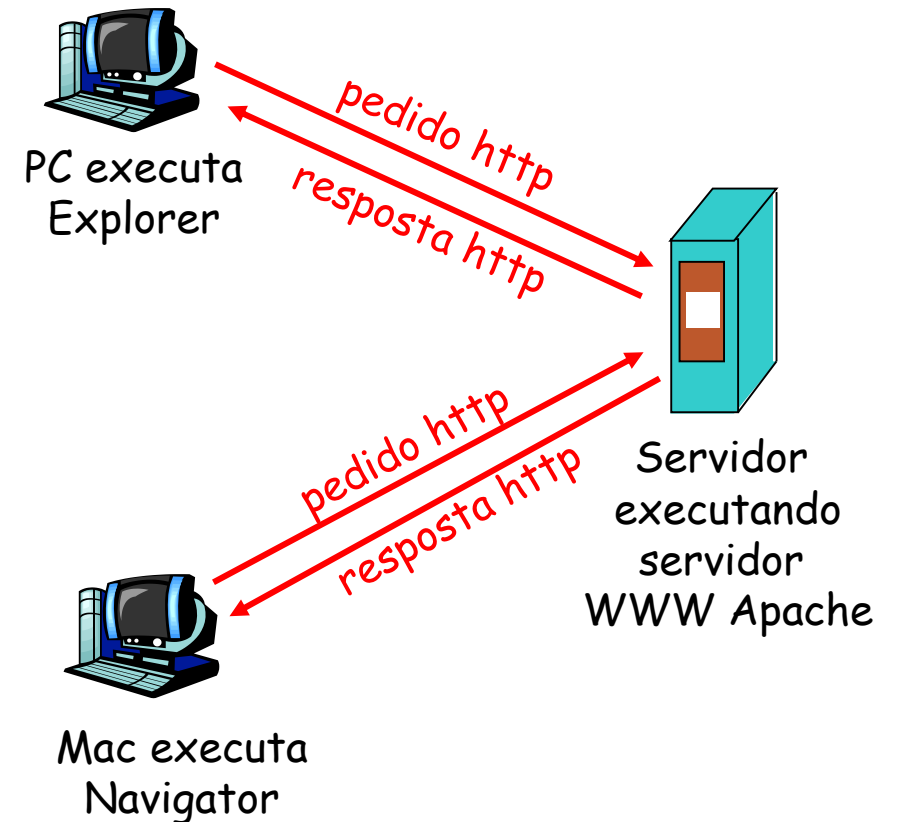
Servidor para WWW é o “servidor Web”:

- Apache HTTP Server (domínio público)
- Internet Information Services (IIS)
- nginx

WWW: o protocolo HTTP

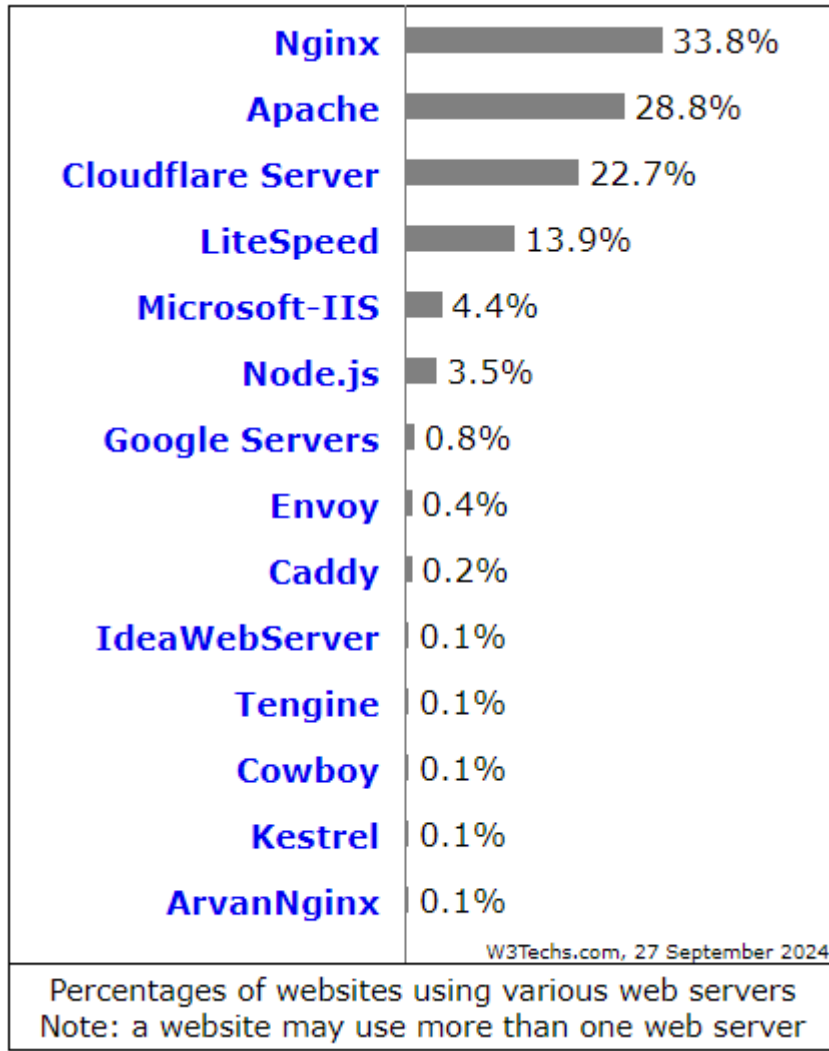
HTTP: Hypertext Transfer Protocol

- protocolo da camada de aplicação para a Web
- Modelo cliente/servidor
 - cliente: browser que pede, recebe, “visualiza” objetos WWW
 - servidor: servidor Web envia objetos em resposta a pedidos
- Versões
 - http1.0: RFC 1945
 - http1.1: RFC 2068
 - http/2: RFC 7540



WWW: o protocolo HTTP

Servidores mais usados



Fonte: https://w3techs.com/technologies/overview/web_server/all

Mais sobre o protocolo HTTP 1.x

Usa serviço de transporte TCP:

- Cliente inicia conexão TCP (cria socket) com servidor, porta 80 (porta default)
- Servidor aceita conexão TCP do cliente
- Mensagens HTTP (mensagens do protocolo da camada de apl) trocadas entre browser (cliente HTTP) e servidor Web (servidor HTTP)
- Encerra conexão TCP

HTTP é “sem estado”

- Servidor não mantém informação sobre pedidos anteriores do cliente
- Simplifica a implementação

Exemplo de Operação http 1.0

Usuário entra com a URL: `www.algumaEscola.edu.br/algumDepto/index.html`

1a. cliente http **inicia conexão TCP** ao servidor http (processo) em `www.algumaEscola.edu.br` Porta 80 é a default para o servidor http .

(contém referência a 10 imagens jpeg)

1b. servidor http no host `www.algumaEscola` esperando pela conexão TCP na porta 80. "aceita" conexão, notificando o cliente

2. cliente http envia **mensagem de pedido** (contendo a URL) para o socket da conexão TCP

3. servidor http recebe mensagem de pedido, forma **mensagem de resposta** contendo o objeto solicitado (`algumDepto/index.index`), envia mensagem para o socket

tempo

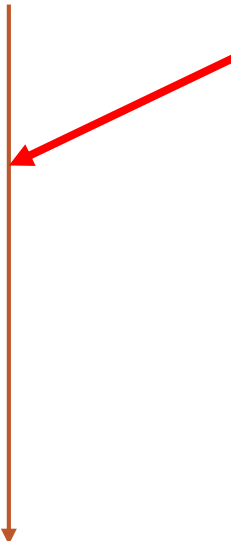
Exemplo (cont.)

5. cliente http recebe mensagem de resposta contendo o arquivo html, apresenta o conteúdo html. Analisando o arquivo html encontra 10 objetos jpeg referenciados

4. servidor http fecha conexão TCP.

6. Passos 1-5 são repetidos para cada um dos 10 objetos jpeg.

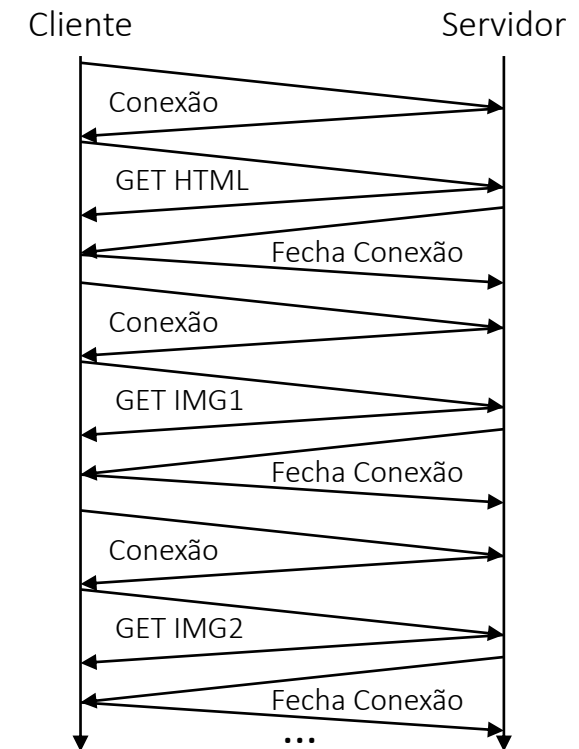
tempo



Conexões Não-persistentes e Persistentes

Conexões não persistentes

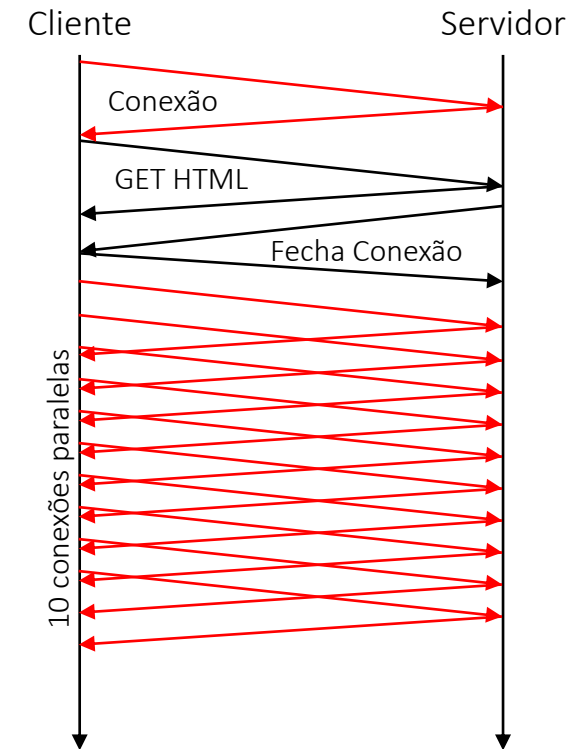
- HTTP/1.0 utiliza conexões não persistentes
- Conexão TCP é fechada após o servidor http enviar o objeto
 - Cada conexão transporta uma requisição e uma resposta
 - No exemplo são necessárias 11 conexões TCP
- Desvantagens: Uma conexão deve ser estabelecida por objeto
 - Necessário alocar buffers no TCP e manter variáveis do TCP tanto no cliente quanto no servidor
 - Atraso na transferência dos objetos
 - Partida lenta do TCP causado pelo controle de congestionamento
 - Conexões podem ser em série ou paralelo
 - Sobrecarrega servidores que atendem vários clientes ao mesmo tempo



Conexões Não-persistentes e Persistentes

Conexões não persistentes

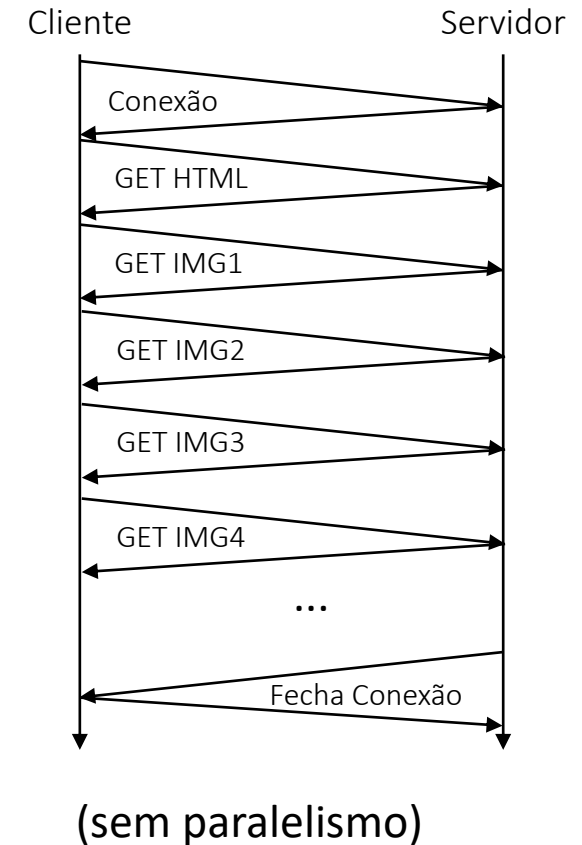
- HTTP/1.0 utiliza conexões não persistentes
- Conexão TCP é fechada após o servidor http enviar o objeto
 - Cada conexão transporta uma requisição e uma resposta
 - No exemplo são necessárias 11 conexões TCP
- Paralelismo reduz algumas desvantagens
 - Reduz latência na carga da página
 - Sofre dos problemas de sobrecarga no servidor e cliente devido a necessidade de buffers
 - Cada conexão sofre da partida lenta: deixa a taxa mais lenta no começo da conexão



Conexões Não-persistentes e Persistentes

Conexões Persistentes (default para o HTTP/1.1)

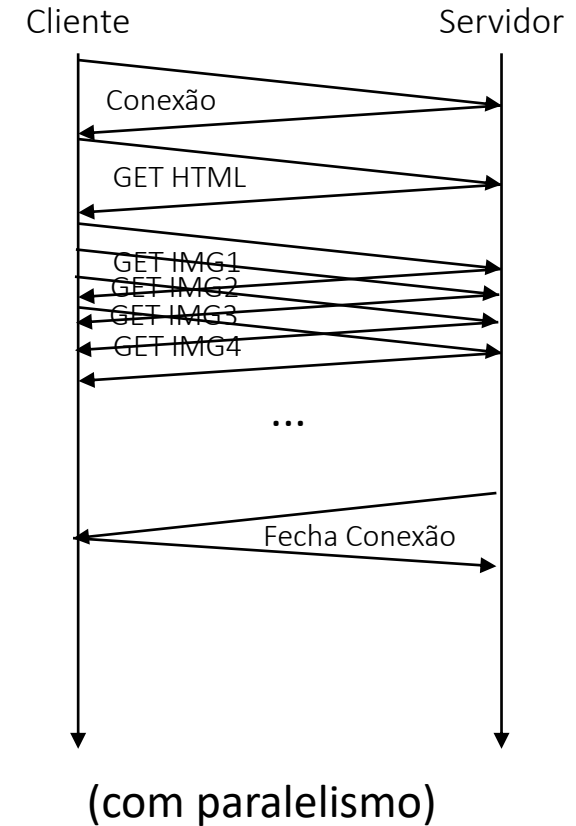
- Servidor deixa a conexão TCP aberta após o envio da resposta
 - Requisições subsequentes são enviadas pela mesma conexão TCP
- Conexão é fechada após um certo tempo de inatividade (configurável)
- Duas versões
 - Sem paralelismo: cliente lança uma nova requisição somente quando a resposta prévia é recebida
 - Com paralelismo: cliente pode lançar pedidos sem respostas prévias (default do HTTP/1.1)



Conexões Não-persistentes e Persistentes

Conexões Persistentes (default para o HTTP/1.1)

- Servidor deixa a conexão TCP aberta após o envio da resposta
 - Requisições subsequentes são enviadas pela mesma conexão TCP
- Conexão é fechada após um certo tempo de inatividade (configurável)
- Duas versões
 - Sem paralelismo: cliente lança uma nova requisição somente quando a resposta prévia é recebida
 - Com paralelismo: cliente pode lançar pedidos sem respostas prévias (default do HTTP/1.1)



Conexões Não-persistentes e Persistentes

Conexões Persistentes (default para o HTTP/1.1)

- Vantagem
 - Reduz o problema de partida lenta do TCP
 - Após ter enviado o primeiro objeto, não tem de enviar o próximo objeto na taxa inicial lenta
 - Reduz o atraso
- Desvantagens de sem paralelismo
 - Conexão TCP fica pendurada enquanto resposta não é recebida
- Vantagens de com paralelismo
 - Conexão fica pendurada por um menor tempo

Modelagem do tempo de resposta

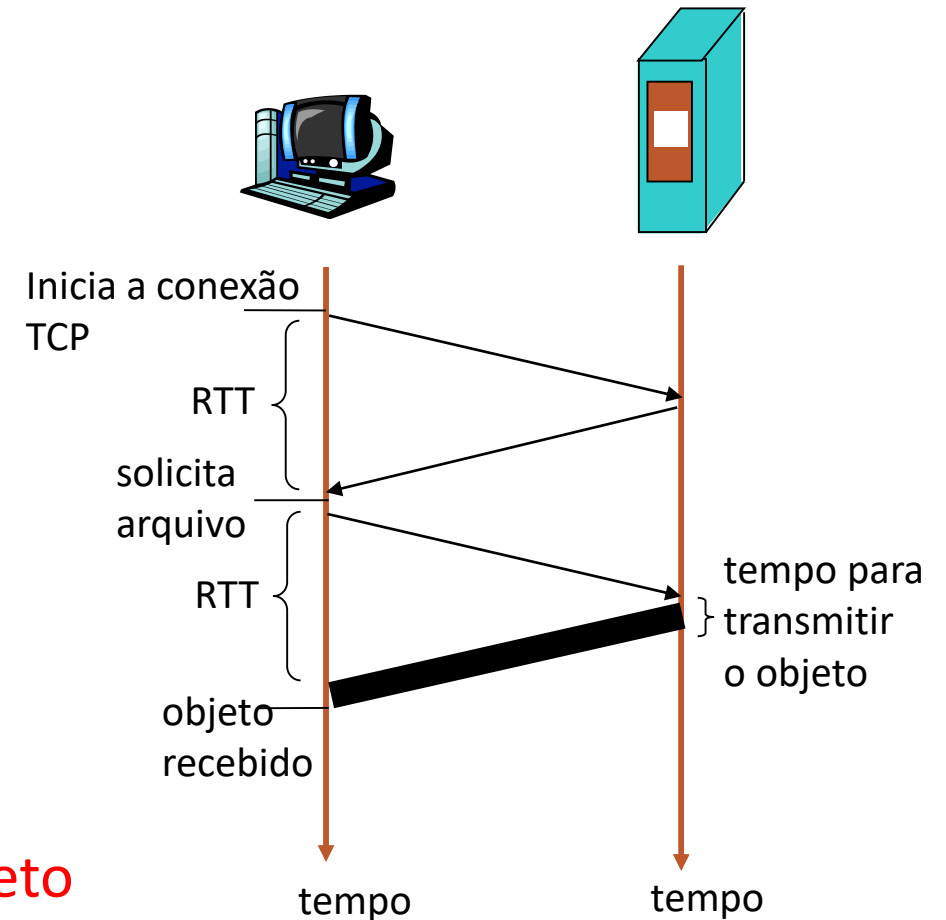
Definição de RTT (Round Trip Time):

- intervalo de tempo entre a ida e a volta de um pacote entre um cliente e um servidor.

Tempo de resposta:

- um RTT para iniciar a conexão TCP
- um RTT para o pedido HTTP e o retorno dos primeiros bytes da resposta HTTP
- tempo de transmissão do objeto (página, imagem, etc.)

$$\text{total} = 2\text{RTT} + \text{tempo de transmissão do objeto}$$



Formato de mensagem HTTP 1.x

Dois tipos de mensagem HTTP: pedido, resposta

Mensagem de pedido HTTP:

- ASCII (formato legível por pessoas)
- Linha de requisição: <método> <URL> <versão do HTTP>

The diagram illustrates the structure of an HTTP request message. It shows a sample request line and header lines, with annotations explaining their components.

linha do pedido
(comandos GET, POST, HEAD)

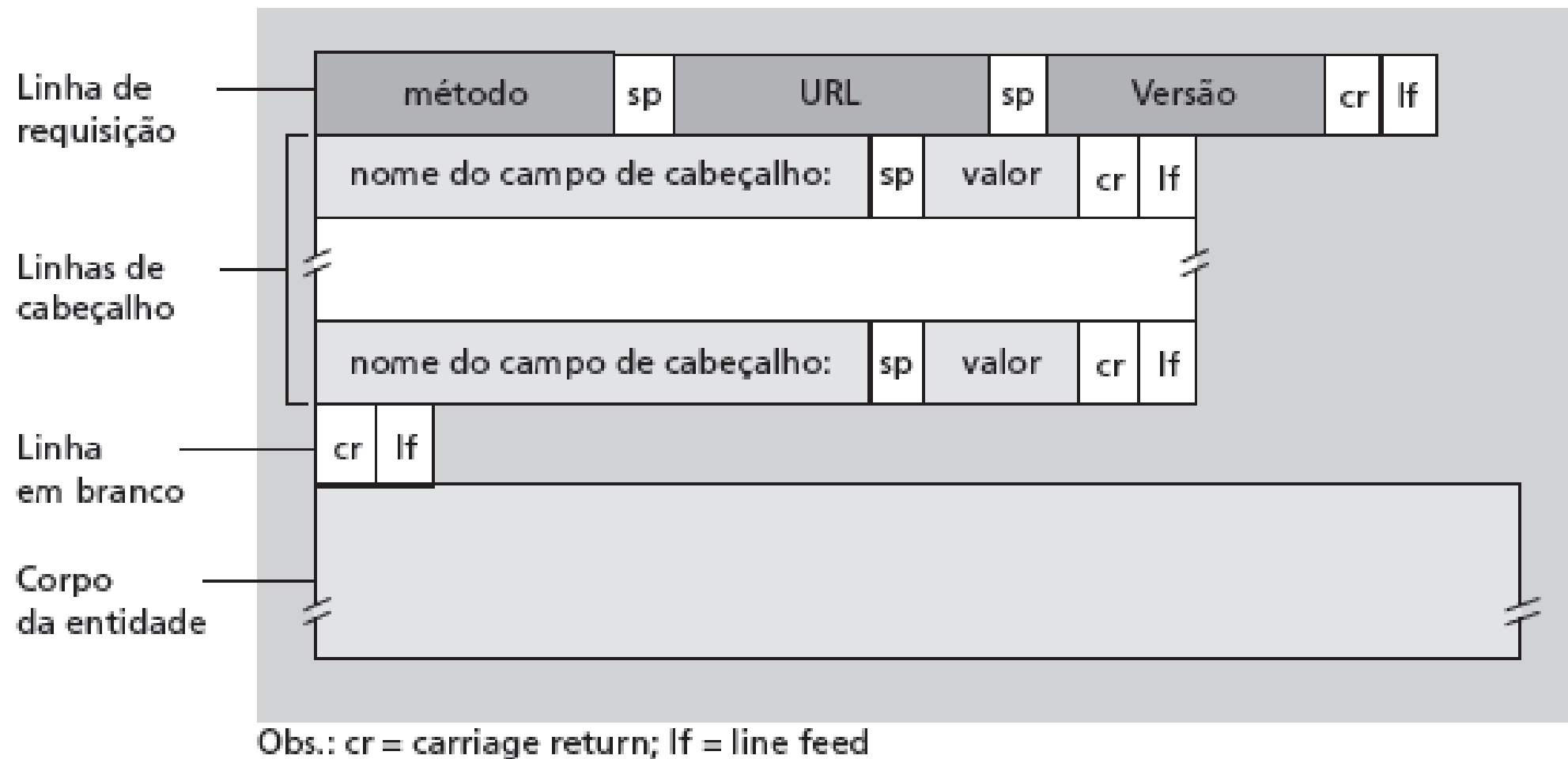
linhas do cabeçalho

```
GET /algumdir/pagina.html HTTP/1.1
Host: www.algumsite.com
Connection: close
User-agent: Mozilla/4.0
Accept-language:br
```

Carriage return, line feed
indicam fim de mensagem

(carriage return, line feed)

Mensagem de Pedido http 1.x: formato Geral



Formato da mensagem de pedido http

HTTP/ 1.0

- Método Get
 - Solicita objeto especificado (URL)
- Método Post
 - Usado pelo cliente http quando o usuário preenche um formulário
 - É uma mensagem de solicitação do objeto
 - Conteúdo específico da página web depende do que o usuário escreveu no formulário
- Método Head
 - Similar ao método Get
 - Mas servidor envia a resposta sem o objeto solicitado
 - Usado para depuração

Formato da mensagem de pedido http

HTTP/1.1

- GET, POST, HEAD
- PUT
 - Envia o arquivo no corpo da entidade para o caminho especificado no campo de URL
- DELETE
 - Apaga o arquivo especificado no campo de URL
- Outros
 - TRACE: Ecoa o pedido, de maneira que o cliente possa saber o que os servidores intermediários estão mudando em seu pedido.
 - OPTIONS: Recupera os métodos HTTP que o servidor aceita.
 - CONNECT: Serve para uso com um proxy que possa se tornar um túnel SSL

Formato da mensagem http 1.x

Método GET

- Linha de comando indica o arquivo a ser transferido ao Browser
- Host indica hospedeiro onde o objeto reside;
- Connection: close indica para servidor usar conexão não persistente
- User-agent: indica o tipo de browser usado
- Accept-language: br indica que usuário prefere uma versão em português do objeto

linhas do
cabeçalho

```
GET /algumdir/pagina.html HTTP/1.1
Host: www.algumsite.com
Connection: close
User-agent: Mozilla/4.0
Accept-language:br
```

(carriage return, line feed)

Formatos HTTP 1.x: Resposta

linha de status
(protocolo
código de status
frase de status)

HTTP/1.1 200 OK

Connection: close

Date: Thu, 06 Aug 1998 12:00:15 GMT

Server: Apache/1.3.0 (Unix)

Last-Modified: Mon, 22 Jun 1998 09:23:24 GMT

Content-Length: 6821

Content-Type: text/html

linhas de
cabeçalho

dados, e.x.,
arquivo html

Objeto solicitado ...

Exemplo de tipo MIME	Descrição
text/plain	Arquivo no formato texto (ASCII)
text/html	Arquivo no formato HTML, utilizado como padrão para documentos Web
Image/gif	Imagem com o formato GIF
Image/jpeg	Imagem com o formato JPEG
application/zip	Arquivo compactado

Formato da mensagem http 1.x: Resposta

Códigos de status (Descritos na RFC 2616)

- 1xx: Informational (Informação) – utilizada para enviar informações para o cliente de que sua requisição foi recebida e está sendo processada;
- 2xx: Success (Sucesso) – indica que a requisição do cliente foi bem sucedida;
 - 200 OK: Pedido com sucesso, objeto pedido está na mensagem
- 3xx: Redirection (Redirecionamento) – informa a ação adicional que deve ser tomada para completar a requisição;
 - 301 Moved Permanently: Objeto pedido movido, nova localização é especificado na mensagem (Location:)
- 4xx: Client Error (Erro no cliente) – avisa que o cliente fez uma requisição que não pode ser atendida;
 - 400 Bad Request: Mensagem de pedido não é entendida pelo servidor
 - 404 Not Found: Documento pedido não foi encontrado no servidor
- 5xx: Server Error (Erro no servidor) – ocorreu um erro no servidor ao cumprir uma requisição válida.
 - 505 HTTP Version Not Supported

Formato da mensagem http 1.x

Método Post

- Usado quando é necessário enviar dados ao servidor
 - para serem processados geralmente por um programa script identificado no Request-URI.
 - informações submetidas são incluídas no corpo da mensagem e formatadas como uma query string, além de conter cabeçalhos adicionais especificando seu tamanho (Content-Length) e seu formato (Content-Type).

```
POST /index.html HTTP/1.0
Accept: text/html
If-modified-since: Sat, 29 Oct 1999 19:43:31 GMT
Content-Type: application/x-www-form-urlencoded
Content-Length: 30
```

```
Nome=NomePessoa&Idade=99&Curso=Computacao
```

HTTP Cliente: faça você mesmo!

1. Telnet para um servidor Web:

```
telnet www.inf.ufsc.br 80
```

Abre conexão TCP para a porta 80 (porta default do servidor http) em www.inf.ufsc.br. Qualquer coisa digitada é enviada para a porta 80 em www.inf.ufsc.br

2. Digite um pedido GET http:

```
GET /~roberto.willrich/ HTTP/1.1  
Host: www.inf.ufsc.br
```

Digitando isto (tecle carriage return duas vezes), você envia este pedido HTTP GET mínimo (mas completo) ao servidor http

3. Examine a mensagem de resposta enviada pelo servidor http!

Pontos Importantes

Protocolo HTTP

- Utiliza o protocolo TCP Porta 80
- Entender os tipos de conexões persistentes e não persistentes
- Pedidos básicos: GET e POST